

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Модели решения задач в интеллектуальных системах»
на тему:

**ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС
МЕТАСИСТЕМЫ OSTIS**

БГУИР КП6 6-05-0611-03 046 ПЗ

Студент гр. 321701
Проверяющий

В. А. Лукашов
Н. В. Зотов

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	6
Введение	7
1 Анализ подходов к разработке интерфейса интеллектуальной системы	8
1.1 Характеристика предметной области	8
1.2 Участники взаимодействия и их задачи	9
1.3 Анализ существующих систем и интерфейсов	10
1.4 Анализ требований к интерфейсу	14
1.5 Вывод по разделу анализа	15
2 Проектирование интеллектуальной системы	17
2.1 Общая архитектура интеллектуальной системы	17
2.2 Архитектура интерфейсного слоя	18
2.3 Модели пользователей и сценарии использования	20
2.4 Проектирование пользовательского интерфейса	26
2.5 Проектирование программного интерфейса	28
2.6 Проектирование интеллектуальных механизмов интерфейса	30
2.7 Вывод по разделу проектирования	32
3 Реализация интеллектуальной системы	34
3.1 Выбор средств реализации	34
3.2 Реализация программного интерфейса	36
3.3 Тестирование интерфейса	39
3.4 Вывод по разделу реализации	41
Заключение	42
Список использованных источников	44

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В курсовой работе используются следующие условные обозначения:

- БЗ — База знаний;
- AI (от англ. Artificial Intelligence) — Искусственный интеллект;
- API (от англ. Application Programming Interface) — Прикладной программный интерфейс;
- CSS (от англ. Cascading Style Sheets) — Каскадные таблицы стилей;
- DOM (от англ. Document Object Model) — Объектная модель документа;
- HTML (от англ. HyperText Markup Language) — Язык разметки гипертекста;
- HTTP (от англ. HyperText Transfer Protocol) — Протокол передачи гипертекста;
- JSON (от англ. JavaScript Object Notation) — Текстовый формат обмена данными, основанный на JavaScript;
- JSX (от англ. JavaScript XML) — Расширение синтаксиса JavaScript;
- OSTIS (от англ. Open Semantic Technology for Intelligent Systems) — Открытая Семантическая Технология Интеллектуальных Систем;
- REST (от англ. Representational State Transfer) — Передача состояния представления;
- SC (от англ. Semantic Code) — Семантический код;
- SCg (от англ. Semantic Code graphical) — Графический семантический код;
- SCn (от англ. Semantic Code natural) — Естественный семантический код;
- SCs (от англ. Semantic Code string) — Семантическая кодовая строка;
- UI (от англ. User Interface) — Пользовательский интерфейс;
- URL (от англ. Uniform Resource Locator) — Уникальный унифицированный адрес;
- UTF-8 (от англ. Unicode Transformation Format, 8-bit) — Формат преобразования Юникода, 8-битная кодировка.

ВВЕДЕНИЕ

Метасистема *OSTIS* представляет собой технологическую основу для построения интеллектуальных систем на базе семантического представления знаний. При практической работе с системой ключевая сложность связана не только с логикой обработки знаний, но и с удобством взаимодействия пользователя с интерфейсом, включая навигацию по структурам базы знаний, выполнение запросов и интерпретацию результатов.

Актуальность темы обусловлена необходимостью повышения удобства и предсказуемости пользовательского интерфейса при работе с существующими компонентами *OSTIS*. Для применения важно, чтобы интерфейс поддерживал типовые пользовательские сценарии без избыточной сложности: переход между режимами представления знаний, выполнение запросов, просмотр результатов, работа с историей действий и структурой разделов.

Цель курсового проекта — разработать и обосновать улучшения пользовательского интерфейса Метасистемы *OSTIS*, обеспечивающие более удобное и устойчивое взаимодействие пользователя с компонентами базы знаний и сервисами интерфейсного слоя.

Задачи курсового проекта:

1 **Доработка интерфейса запросов AskAI.** Реализовать корректную обработку пользовательских запросов и отображение ответов в интерфейсе, включая улучшение качества обратной связи.

2 **Улучшение навигации и режимов представления.** Обеспечить предсказуемый переход между режимами *SCn/SCg* и улучшить навигационные сценарии при работе с элементами базы знаний.

3 **Доработка компонентов истории и разделов.** Исправить проблемы отображения и операций добавления/удаления в связанных интерфейсных модулях.

4 **Повышение согласованности интерфейса.** Привести элементы управления и визуальные состояния к единому стилю, включая корректную работу в светлой и тёмной темах.

Объект исследования. Пользовательский интерфейс интеллектуальной системы, реализуемой на базе Метасистемы *OSTIS*.

Предмет исследования. Методы и средства проектирования и доработки интерфейсного слоя веб-приложения *react-sc-web*, обеспечивающего взаимодействие пользователя с компонентами базы знаний и сервисами системы.

Границы проекта. В рамках курсового проекта рассматриваются проектирование и реализация изменений интерфейсного слоя и связанных клиент-серверных точек взаимодействия. Реализация внутренних механизмов логического вывода и полная переработка ядра Метасистемы *OSTIS* в задачи проекта не входит.

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕРФЕЙСА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

1.1 Характеристика предметной области

Разрабатываемый пользовательский интерфейс функционирует в предметной области интеллектуальных систем и технологий представления знаний. В качестве базовой платформы используется Метасистема *OSTIS* [1], предназначенная для построения, сопровождения и развития интеллектуальных систем на основе формализованных семантических моделей [2]. Данная предметная область характеризуется высокой степенью абстракции, использованием онтологических структур и необходимостью точного соответствия внутреннего представления знаний установленным стандартам.

Особенностью рассматриваемой области является применение низкоуровневых форм представления знаний, ориентированных прежде всего на машинную обработку. В связи с этим одной из ключевых задач становится преобразование сложных семантических конструкций в формы, доступные для восприятия и использования человеком. Пользовательский интерфейс в данном контексте выполняет роль средства абстрагирования, обеспечивая переход от формального описания знаний к наглядному и управляемому представлению.

В рамках Метасистемы *OSTIS* решаются задачи, связанные с навигацией по структурам базы знаний, анализом семантических связей между объектами, а также формированием и выполнением пользовательских запросов. Интерфейс должен поддерживать поиск и просмотр компонентов системы, отображение фрагментов базы знаний в различных формах и взаимодействие с интеллектуальными механизмами обработки информации. Ожидаемым результатом функционирования системы является повышение эффективности работы пользователей и упрощение процесса взаимодействия с интеллектуальными компонентами.

При проектировании интерфейса необходимо учитывать ряд ограничений и особенностей предметной области. К ним относятся требования к производительности, выражающиеся в допустимом времени отклика на действия пользователя и обработке запросов, а также необходимость строгого соблюдения стандартов представления и обмена данными, принятых в технологии *OSTIS*. Дополнительно учитываются требования к доступности и универсальности интерфейса, включая его корректную работу в различных программных и аппаратных средах, а также адаптацию к различным форматам отображения.

1.2 Участники взаимодействия и их задачи

Взаимодействие пользователей с интеллектуальной системой, реализованной на базе Метасистемы *OSTIS*, осуществляется в рамках распределённой архитектуры с разделением клиентской и серверной частей. В процессе функционирования системы задействованы несколько категорий участников, каждая из которых выполняет определённые функции и использует интерфейс в собственном контексте.

Разработчик интеллектуальных систем относится к категории конечных пользователей и использует систему в процессе создания и сопровождения компонентов базы знаний (*БЗ*). Основной целью данного участника является формирование, анализ и модификация семантических структур. В рамках своей деятельности разработчик осуществляет навигацию по библиотеке компонентов, просматривает и редактирует фрагменты *БЗ* в различных формах представления, а также формирует запросы к системе, включая запросы на естественном языке. В качестве входных данных используются семантические конструкции и пользовательские команды, а результатом взаимодействия являются визуализированные структуры и ответы интеллектуальных механизмов. Контекст использования интерфейса — рабочее место разработчика с доступом к системе через веб-браузер.

Администратор метасистемы также является конечным пользователем, однако его взаимодействие с системой носит управленческий характер. Целью администратора является обеспечение корректного функционирования системы, контроль изменений и управление правами доступа. В процессе работы администратор задаёт параметры доступа, рассматривает предложения на изменение компонентов базы знаний и анализирует информацию о состоянии системы. Интерфейс для данной роли используется в режиме административной панели и предполагает работу в защищённой среде с повышенными требованиями к надёжности и безопасности.

Платформа *OSTIS* выступает в роли внутренней подсистемы, обеспечивающей хранение знаний и выполнение интеллектуальной обработки. Она отвечает за управление внутренним представлением знаний, интерпретацию моделей пользовательского интерфейса и выполнение запросов, поступающих от клиентской части. В качестве входных данных платформа получает команды и запросы от интерфейса, а в качестве выходных — структурированные результаты обработки, предназначенные для последующей визуализации. Работа данной подсистемы осуществляется в серверной среде и не предполагает прямого взаимодействия с пользователем.

Клиентское устройство пользователя представляет собой техническое средство доступа к системе. Его основной задачей является отображение элементов пользовательского интерфейса, обработка пользовательских действий и передача соответствующих запросов на сервер. В рамках кли-

ентского окружения выполняется рендеринг интерфейсных компонентов, обработка событий ввода и визуализация полученных данных. Контекст использования включает современные веб-браузеры на настольных и мобильных устройствах.

Таким образом, основные сценарии взаимодействия реализуются через веб-интерфейс, в котором клиентская часть обеспечивает ввод и визуальное представление информации, а серверная — интеллектуальную обработку и управление знаниями. Конечные пользователи взаимодействуют с системой опосредованно, посредством интерфейсных компонентов, транслирующих их действия во внутренние операции Метасистемы *OSTIS*.

1.3 Анализ существующих систем и интерфейсов

Обзор аналогичных систем

Для выбора оптимальных архитектурных и интерфейсных решений Метасистемы *OSTIS* был выполнен анализ существующих программных продуктов, применяемых для визуализации данных, автоматизации рабочих процессов и управления персональными БЗ. Основной целью исследования стало сравнение инструментов, предлагающих различные методы организации, управления и представления семантических и структурированных данных. В качестве наиболее подходящих аналогов были выделены системы *Flowise*, *n8n* и *Logseq*, которые обеспечивают эффективное построение потоков данных, моделирование процессов и управление связями между объектами знаний. *Flowise* предоставляет веб-интерфейс, позволяющий пользователю визуально формировать последовательности действий посредством перетаскивания блоков и соединения их логическими связями, обеспечивая наглядное отображение процессов и ориентируясь на взаимодействие человек–система. *n8n* является open-source платформой для автоматизации процессов, предоставляющей возможность создавать и управлять цепочками операций между различными сервисами через графический веб-интерфейс, что облегчает интеграцию и визуализацию потоков данных. *Logseq* представляет собой персональную систему управления знаниями с блоковой структурой и двунаправленными связями между заметками, обеспечивая гибкую организацию информации и визуализацию графа знаний через веб- и desktop-приложения.

Анализ интерфейсов существующих решений

Детальное изучение пользовательских интерфейсов выбранных систем позволяет выявить характерные подходы к организации взаимодействия с данными, а также преимущества и ограничения, которые важно учитывать при проектировании интерфейса Метасистемы.

Интерфейс системы Flowise

Flowise [3] предоставляет веб-интерфейс (рисунок 1.1), ориентирован-

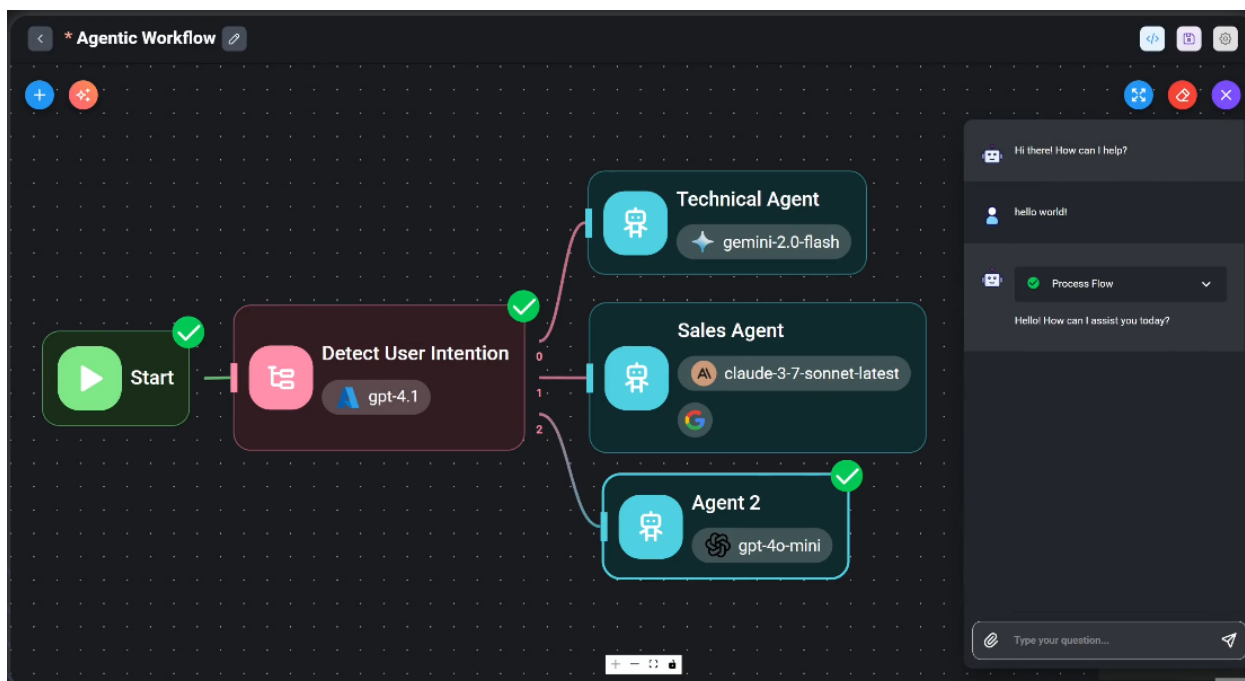


Рисунок 1.1 – Интерфейс платформы Flowise

ный на визуальное построение потоков данных и интеграцию различных сервисов. Основной рабочий экран системы организован по принципу «drag-and-drop», где пользователь располагает блоки операций на рабочей области и соединяет их логическими связями. Структура интерфейса включает панель доступных блоков, рабочую область для построения схем, а также панель свойств выбранного блока для настройки параметров. Ключевым сценарием взаимодействия является построение цепочек обработки данных, от источника до конечного результата, с возможностью тестирования и отладки на каждом этапе. Основными достоинствами интерфейса являются высокая наглядность процессов, простота перетаскивания элементов и возможность быстрого прототипирования. Однако среди недостатков можно отметить ограниченные возможности визуализации больших потоков данных и потенциальную перегруженность интерфейса при создании сложных схем.

Интерфейс системы n8n

Система n8n [4] представляет собой open-source платформу для автоматизации рабочих процессов, предоставляющую пользователю графический веб-интерфейс (рисунок 1.2). Основной рабочий экран содержит панель узлов, область построения цепочек действий и контекстную панель настроек каждого узла. Пользователь взаимодействует с системой через соединение блоков, настройку триггеров и параметров операций, а также тестирование выполнения процессов. Среди ключевых сценариев — интеграция внешних сервисов, автоматизация задач и управление потоками данных. Сильными сторонами интерфейса являются интуитивно понятное

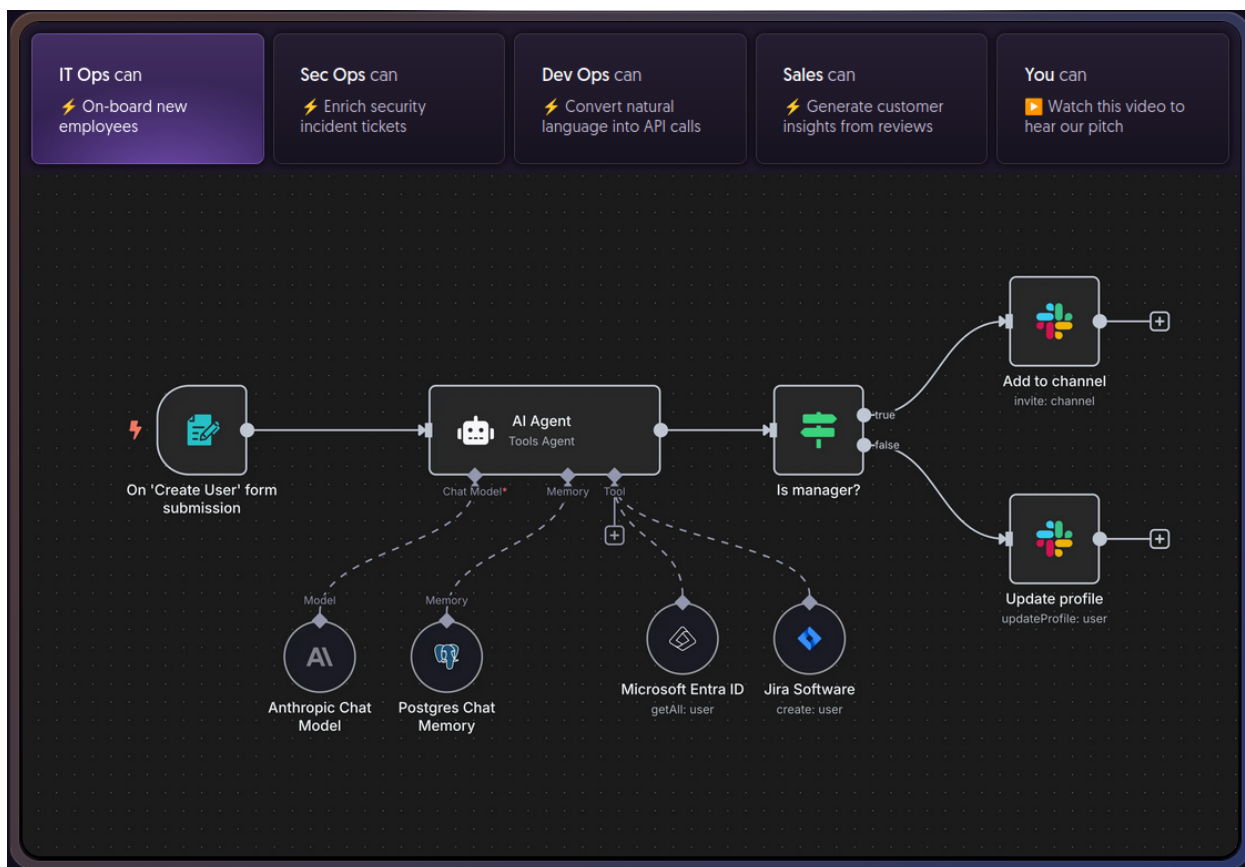


Рисунок 1.2 – Интерфейс системы n8n

построение процессов, прозрачная настройка узлов и наличие встроенных инструментов для отладки. Среди ограничений можно отметить необходимость понимания логики потоков данных для корректного соединения узлов, а также сравнительно высокую сложность работы с масштабными схемами без использования дополнительных визуальных фильтров.

Интерфейс системы Logseq

Logseq [5] является персональной БЗ (рисунок 1.3) с поддержкой блочной структуры и двунаправленных ссылок между заметками. Основной рабочий экран включает панель навигации, область редактирования блоков и граф знаний, отображающий связи между заметками. Типовые операции пользователя включают создание и связывание заметок, группировку информации по проектам, поиск и фильтрацию данных, а также визуализацию графа для анализа семантических связей. Ключевой особенностью интерфейса является гибкость организации информации и возможность персонализации структуры базы знаний под конкретные задачи. Сильными сторонами являются наглядная визуализация связей, возможность интеграции с Markdown и открытость платформы. К недостаткам можно отнести ограниченную автоматизацию процессов и необходимость привыкания к блочной структуре для пользователей, ранее не работавших с подобными системами.



Рисунок 1.3 – Интерфейс персональной базы знаний Logseq

Сравнительный анализ

Проведённый анализ показывает, что рассмотренные решения покрывают отдельные аспекты требований, предъявляемых к Метасистеме *OSTIS*, но ни одно из них не обеспечивает комплексного сочетания визуальной наглядности, гибкости управления знаниями и поддержки семантических связей. *Flowise* и *n8n* демонстрируют высокую наглядность процессов и потоков данных, позволяя пользователю быстро создавать и тестировать последовательности действий, однако их интерфейсы ориентированы преимущественно на моделирование процессов и автоматизацию, а не на управление семантическими связями и построение БЗ. В то же время *Logseq* обеспечивает гибкую организацию информации, поддержку двунаправленных ссылок и визуализацию графа знаний, но ограничен в возможностях автоматизации потоков и интеграции с внешними сервисами.

Интерфейс Метасистемы *OSTIS* должен объединять сильные стороны этих подходов: обеспечивать визуальную наглядность потоков и структур данных, предоставлять гибкие инструменты для управления связями между объектами знаний и поддерживать интерактивное построение процессов. Уникальными возможностями разрабатываемой системы станут интеграция инструментов автоматизации и визуального моделирования с семантической строгостью работы с типизированными связями SC-кода[1], поддержка коллективного редактирования с ролевой моделью и встроенные средства анализа и генерации дочерних систем, что позволит устранить ограничения

существующих аналогов и повысить эргономику веб-интерфейса.

1.4 Анализ требований к интерфейсу

Функциональные требования

Исходя из анализа предметной области и изучения интерфейсов существующих систем сформулированы требования к пользовательскому интерфейсу Метасистемы *OSTIS*. Интерфейс должен обеспечивать выполнение операций для различных ролей пользователей, таких, как Разработчик, Администратор и Пользователь.

1 Авторизация и управление доступом. Интерфейс должен обеспечивать форму входа в систему, регистрацию новых пользователей, управление правами доступа и отображение текущего статуса аутентификации. Для Администратора необходимо предусмотреть возможность назначения ролей и контроля уровня доступа к компонентам базы знаний.

2 Автоматизация процессов и интеграция. Интерфейс должен обеспечивать инструменты для визуального моделирования процессов и потоков данных (по примеру *Flowise* и *n8n*), включая создание цепочек операций, настройку параметров и тестирование выполнения. Пользователь должен иметь возможность интегрировать внешние сервисы и просматривать результаты автоматизированных сценариев в наглядной форме.

3 Навигация. Интерфейс должен предоставлять пользователю удобные средства навигации по структуре базы знаний, поддерживать поиск элементов по различным критериям (название, тип, семантические связи) и обеспечивать переход по связанным объектам без потери контекста.

4 Управление семантическими связями и графом знаний. Интерфейс должен предоставлять средства для создания и редактирования связей между объектами знаний, визуализации графа связей (по примеру *Logseq*) и фильтрации информации для анализа. Это необходимо для поддержки гибкой структуры базы знаний и эффективного взаимодействия с семантическими данными.

Нефункциональные требования

Интерфейс также должен удовлетворять и следующим нефункциональным требованиям:

1 Скорость работы и отзывчивость. Интерфейс должен обеспечивать быстрый отклик на действия пользователя (клик, перемещение, переключение режимов), выполнение сложных процессов и отображение графов знаний. Не допускается зависание или блокировка интерфейса при обработке больших потоков данных.

2 Доступность и кроссплатформенность. Интерфейс следует обеспечивать в современных веб-браузерах (Chrome, Firefox, Microsoft Edge) [6]. Не допускается использование технологий, требующих установки специ-

фических плагинов или расширений.

3 Безопасность и конфиденциальность. Следует обеспечить защиту данных пользователей и компонентов базы знаний, включая разграничение прав доступа и ведение журналов изменений. Не допускается хранение паролей и конфиденциальной информации в открытом виде.

Требования к «интеллектуальности» интерфейса

Интерфейс Метасистемы *OSTIS* должен включать элементы интеллектуального поведения, обеспечивающие помощь пользователю в работе с *БЗ* и визуальными структурами:

1 Контекстная поддержка действий. Интерфейс должен предоставлять рекомендации и подсказки на основе текущего состояния элементов *БЗ* и действий пользователя. Например, при редактировании узлов или связей система должна предлагать допустимые варианты операций с учётом структуры графа.

2 Объяснение результатов операций. При выполнении сложных действий (например, автоматической генерации связей или проверки согласованности данных) интерфейс должен отображать краткое пояснение, поясняющее пользователю, почему был выбран тот или иной результат, на основе семантических правил.

3 Адаптация интерфейса под пользователя. Система должна сохранять индивидуальные настройки отображения и предпочтения пользователя, например, вид графа или степень детализации информации. Интерфейс должен корректно подстраиваться под задачи конкретной роли (Разработчик, Администратор, Пользователь).

4 Интеллектуальная обработка сообщений. Интерфейс должен предоставлять уведомления и рекомендации в случаях ошибок или некорректных действий пользователя, формируя их на основе анализа текущего контекста и правил работы с *БЗ*.

1.5 Вывод по разделу анализа

Проведённое исследование предметной области показало, что одной из основных трудностей при работе с Метасистемой *OSTIS* является сложность взаимодействия с семантическими структурами и потоками данных. Для пользователей важно создать интерфейс, который скрывает детали низкоуровневого SC-кода [7] и обеспечивает удобный доступ к информации в базе знаний. Специализированный пользовательский интерфейс становится ключевым инструментом для упрощения этих процессов и повышения эффективности работы с системой.

В процессе анализа участников взаимодействия были выделены основные роли: Разработчик, Администратор и Пользователь. Для каждой роли определены критически важные сценарии работы, включая авториза-

цию и управление учетными записями, поиск и навигацию по базе данных, редактирование и визуализацию элементов знаний, а также поддержку совместной работы и управления потоками данных.

Обзор существующих аналогов (*Flowise*, *n8n* и *Logseq*) показал, что ни одно решение не объединяет все необходимые возможности одновременно. *Flowise* и *n8n* предоставляют удобные средства для визуального построения и тестирования процессов, но их возможности по управлению связями и структурой знаний ограничены. *Logseq* позволяет эффективно работать с графом знаний и двунаправленными связями, однако не обеспечивает инструментов для автоматизации процессов и интеграции внешних сервисов.

На основании полученных данных при проектировании интерфейса Метасистемы *OSTIS* будут применены современные веб-технологии с модульной архитектурой, реализованы инструменты для визуального моделирования процессов и автоматизации операций, предусмотрена поддержка адаптивных подсказок и механизмов коллективного редактирования с ограничением прав пользователей. Эти решения позволят устранить недостатки существующих аналогов, повысить наглядность и удобство работы с БЗ и обеспечить более эффективное взаимодействие с системой для всех категорий пользователей.

2 ПРОЕКТИРОВАНИЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы

Интеллектуальная система *OSTIS*, рассматриваемая в рамках проекта, реализуется как многокомпонентная веб-система с разделением на уровень представления, серверный уровень обработки запросов и уровень семантического ядра. Такой подход выбран как практически применимый для существующего кода **react-sc-web** и серверной части **sc-web**, а также как наиболее устойчивый к локальным доработкам интерфейса без изменения ядра *SC-памяти*.

База знаний выступает центральным компонентом архитектуры и хранит данные в виде семантической сети. Работа с этой сетью выполняется не напрямую из интерфейса, а через серверные обработчики и вызовы команд *SC-технологии*. Это решение позволяет отделить пользовательские сценарии от низкоуровневых операций с семантическими структурами и тем самым снизить риск нарушения целостности данных при развитии интерфейса.

Подсистема обработки запросов расположена между веб-интерфейсом и БЗ. На её стороне выполняются маршрутизация запросов, проверка входных данных, вызов необходимых команд к ядру системы и формирование ответа, пригодного для отображения в *UI*. Такое разделение оказалось критически важным в ходе реализации: часть исправлений (например, логика ответов **AskAI** и обработка ошибок) потребовала изменений именно в серверных обработчиках, а не только в клиентском коде.

Пользовательский интерфейс реализован как веб-приложение и отвечает за визуализацию, навигацию, ввод пользовательских данных и отображение результатов. Интерфейсный слой не реализует вычислительную логику семантического вывода: он инициирует операции и интерпретирует полученный результат для пользователя. В рамках проекта это проявляется в доработках сценариев **AskAI**, истории, навигации по разделам, переходов между режимами *SCn/SCg* и визуальной обратной связи при ошибках.

При выборе архитектурного подхода рассматривались два варианта. Первый — «толстый клиент» с переносом значительной части логики обработки в браузер. Второй — сохранение тонкого клиентского слоя с концентрацией вычислительной и интеграционной логики на сервере. Выбран второй вариант, поскольку он лучше согласуется с текущей архитектурой *OSTIS*, упрощает контроль корректности операций с БЗ и уменьшает связанность интерфейсного кода с внутренними форматами данных ядра.

Для повышения обоснованности архитектурных решений используются две взаимодополняющие диаграммы.

Во-первых, BPMN-диаграмма (рисунок 2.2) фиксирует процессный аспект архитектуры: как пользовательское действие инициирует последовательность шагов в интерфейсном слое, передаётся в серверный контур, обрабатывается с обращением к компонентам семантического ядра и возвращается в виде результата для отображения в *UI*.

Во-вторых, компонентная диаграмма (рисунок 2.3) отражает структурный аспект: состав ключевых программных компонентов интерфейсного слоя и их связи с коммуникационным контуром и внутренними сервисами.

Совместное использование этих диаграмм позволяет одновременно обосновать динамику выполнения сценариев и статическую организацию подсистем, что необходимо для целостного описания общей архитектуры.

Таким образом, интерфейс в общей архитектуре занимает верхний уровень и обеспечивает управляемый доступ к функциям системы через серверный слой, а не прямое взаимодействие с ядром. Это решение обеспечивает технологическую совместимость с существующей платформой и позволяет развивать интерфейс итеративно без пересборки архитектуры интеллектуального ядра.

2.2 Архитектура интерфейсного слоя

Структура модулей и слоёв

Интерфейс **react-sc-web** спроектирован по слоевой модели, в которой разделены представление, управление состоянием, прикладная логика взаимодействия и коммуникационный контур. Такое разделение было выбрано не декларативно, а исходя из характера исправлений, выполненных в реализации: большая часть дефектов устранялась локально в конкретном слое без каскадной переработки всей системы [8].

Слой представления реализован на React-компонентах и отвечает за отображение данных, маршрутов и состояний интерфейса. В этом слое были выполнены изменения, напрямую влияющие на пользовательский опыт: корректировка визуального поведения элементов истории, настройка режимов отображения *SCn/SCg*, доработка внешнего вида кнопок и контекстных элементов, а также улучшения в диалоговом блоке **AskAI**.

Слой состояния базируется на **Redux Toolkit** [9] и обеспечивает согласованное хранение данных, которые используются в нескольких частях приложения. Выбор централизованного хранилища подтверждён практикой реализации: исправления, связанные с историей действий, текущими маршрутами и переключениями режимов, потребовали предсказуемого управления состоянием, которое проще обеспечивать через единый контур, чем через разрозненные локальные состояния компонентов.

Слой логики взаимодействия реализован через сервисные функции и хуки [10], которые связывают пользовательские действия с вызовами *API*

и навигацией. Здесь сосредоточены сценарии, где важно предотвратить побочные эффекты (дубли запросов, некорректные повторные переходы, конфликт фильтрации и поиска). В ходе проекта именно этот слой обеспечил корректную декомпозицию сложных *UI-сценариев* без переноса бизнес-логики в *JSX*.

Коммуникационный слой использует **axios** для *HTTP-обмена* с сервером и **ts-sc-client** для операций, связанных с семантическими сущностями. Такой гибридный подход выбран как компромисс между универсальностью *REST-взаимодействия* и необходимостью работать с предметно-специфичными операциями *SC-технологии*. Альтернативой мог бы быть единый унифицированный *REST-контур* без клиентских вызовов **ts-sc-client**, однако в текущей версии проекта это привело бы к росту серверной обвязки и потере части уже существующих интеграционных возможностей.

С практической точки зрения архитектура интерфейсного слоя подтверждается тем, что реализованные доработки легли в существующие границы ответственности: визуальные проблемы исправлялись в компонентах и стилях, ошибки сценариев — в логике взаимодействия, нестабильность обмена — в обработке коммуникационных ошибок. Это позволило сохранить связность проекта и избежать «точечного переписывания» ядра *UI*.

Взаимодействие с внутренними компонентами

Взаимодействие **react-sc-web** с внутренними компонентами системы строится по двум основным каналам: *HTTP-запросы* к серверным обработчикам и операции, связанные с семантическими сущностями, через **ts-sc-client**. На практике это разграничение позволило адаптировать поведение интерфейса под фактические ответы сервера без разрыва существующих пользовательских сценариев.

Для пользовательских операций уровня приложения (история, навигационные действия, операции с разделами, сервисные вызовы) используется *HTTP-контур* с сериализацией данных в *JSON*. Для операций, где требуется адресная работа с элементами семантической сети, применяются специализированные вызовы, обеспечивающие извлечение и интерпретацию данных в терминах *SC-адресов* и связанных структур. Обработка отказов реализована каскадно: сначала фиксируется ошибка транспортного уровня (недоступность сервера, тайм-аут, сетевой сбой), затем отдельно проверяется корректность прикладного результата. В интерфейсе это сопровождается сообщением, не разрывающим пользовательский сценарий. Такой подход был выбран как более устойчивый по сравнению с «жёстким падением» сценария, поскольку в рамках проекта важно сохранять работоспособность интерфейса даже при частичной недоступности отдельных операций.

Важно подчеркнуть, что в текущей реализации не вводились искусственно усложнённые механизмы оркестрации обмена: архитектура опи-

рается на уже существующий стек и расширяется эволюционно. Это соответствует цели курсового проекта — показать обоснованные инженерные решения в границах реальной кодовой базы, а не демонстрировать абстрактно «идеальную» архитектуру, не подтверждённую реализацией.

2.3 Модели пользователей и сценарии использования

Ролевые модели пользователей и внешних систем

Для проектирования интерфейсного слоя **react-sc-web** целесообразно выделить две прикладные роли, отличающиеся не абстрактным статусом, а характером фактических действий в системе. Такой подход выбран потому, что в рамках проекта дорабатывался прежде всего пользовательский контур взаимодействия с существующей инфраструктурой *OSTIS*, а не самостоятельная подсистема полномасштабного администрирования. Первая роль — пользователь-исследователь — ориентирована на ежедневную работу с содержимым базы знаний: поиск и просмотр элементов, навигацию по разделам, выполнение запросов через **AskAI**, анализ результатов в режимах *SCn/SCg*, работу с историей действий и переходами между связанными сущностями. Вторая роль — пользователь с расширенными полномочиями сопровождения (условно «администратор интерфейса») — помимо перечисленных сценариев использует операции, связанные с сопровождением структуры интерфейсной среды, в том числе работу с библиотекой компонентов и контролем корректности выполнения соответствующих действий в *UI*.

Выбор именно такой ролевой модели обусловлен тем, что она напрямую соответствует реализованным и проверенным сценариям: в проекте подтверждены доработки навигации, **AskAI**, истории и операций с разделами, а также исправления в библиотеке компонентов; при этом функции глубокого системного администрирования уровня ядра *SC-технологии* (включая модификацию внутренних протоколов или выделенную политику многоконтурной аутентификации) в данной реализации не являлись целевыми и поэтому не включаются в активный ролевой контур проекта. Альтернативный вариант — детализировать 3–4 роли с отдельными административными ветками — был рассмотрен, но отвергнут как методически избыточный для фактического объёма реализованного функционала и как создающий разрыв между текстом проектирования и реальными результатами раздела реализации.

Внешними участниками взаимодействия при такой постановке выступают серверный слой **sc-web** и семантическое ядро *OSTIS*. Интерфейсный клиент передаёт пользовательские действия в серверный контур по *HTTP*-каналу и получает данные для отображения, а операции, связанные с семантическими сущностями, выполняются через предусмотренные

механизмы работы с *SC*-структурами. Это разграничение позволяет интерпретировать роль пользователя именно как роль инициатора сценариев и потребителя результата, а роль внутренних подсистем — как роль исполнителя вычислительных и семантических операций. Тем самым достигается связность между моделью ролей, архитектурой раздела 2.2 и фактическими изменениями, описываемыми в реализации.

На рисунке 2.1 представлена диаграмма прецедентов [11], которая фиксирует распределение основных пользовательских сценариев по указанным ролям. В контексте выполненной работы диаграмма интерпретируется не как формальная «полная» модель прав всей платформы, а как рабочая модель доступа к тем сценариям, которые были предметом проектирования и последующей доработки в интерфейсном слое.

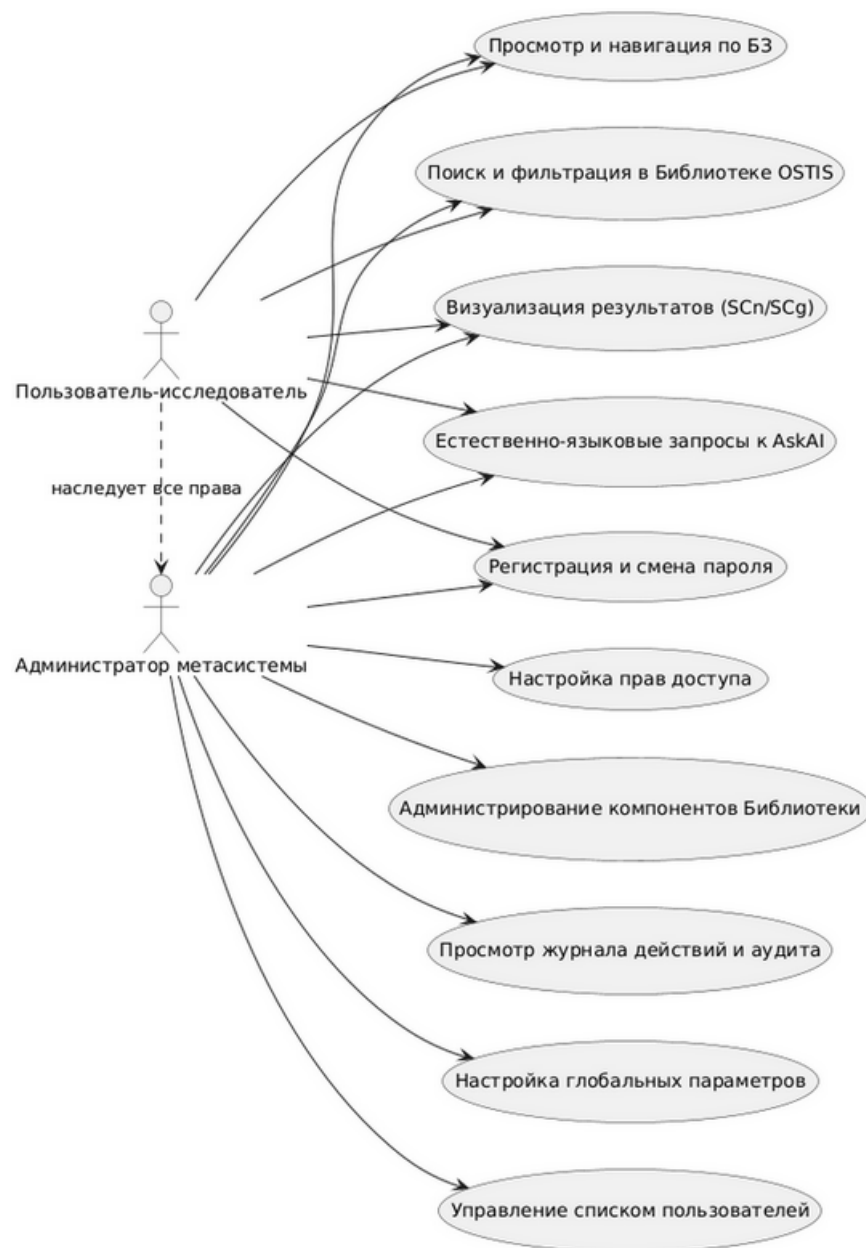


Рисунок 2.1 – Диаграмма прецедентов: основные роли и их доступ к функциям системы

Сценарии использования

Сценарии использования в данном проекте описываются как последовательности действий пользователя в интерфейсном слое с явной фиксацией реакций системы и точек, в которых проявляются архитектурные решения, принятые в разделе проектирования. Такой способ представления выбран осознанно: он обеспечивает связность между постановкой задач, реализованными изменениями и критериями проверки результата, а также позволяет использовать диаграммы как инструмент объяснения логики системы, а не как формальное дополнение к тексту.

Сценарий навигации по разделам. Пользователь открывает раздел по прямой ссылке или переходит в него из интерфейса. Система определяет текущий контекст, строит навигационный путь и отображает его в виде цепочки переходов. Промежуточные элементы цепочки доступны для перехода, что позволяет вернуться на нужный уровень иерархии без повторного поиска. Если открыт объект, не являющийся разделом декомпозиции, система отображает корректный упрощённый контекст без ложной интерактивности. Этот сценарий отражает реальный результат доработки: улучшение ориентирования пользователя в структуре БЗ при сохранении согласованности маршрутизации и отображения.

BPMN-диаграмма на рисунке 2.2 используется для обоснования именно процессной стороны решения: она показывает, как инициирующее действие пользователя последовательно проходит через этапы интерпретации URL, получения структуры разделов, выбора ветви обработки и формирования интерфейсного результата. Включение этой диаграммы в описание сценария важно потому, что она демонстрирует не только «что отображается», но и «почему система ведёт себя именно так» при альтернативных входных условиях.

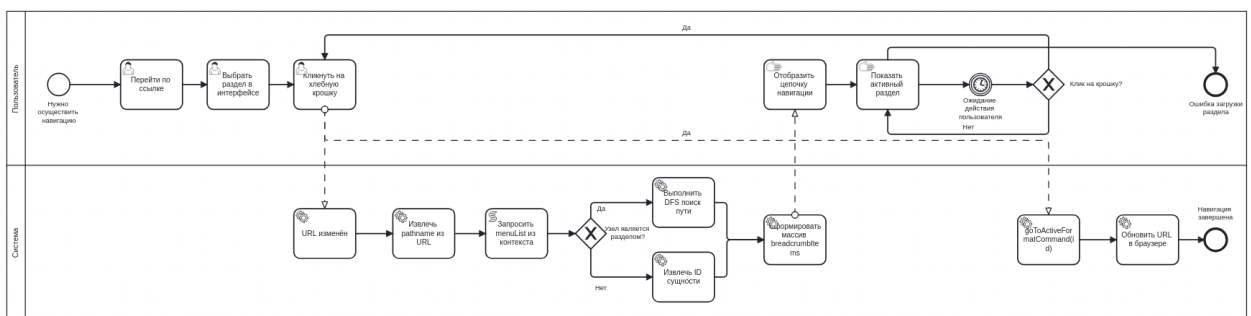


Рисунок 2.2 – BPMN-диаграмма: бизнес-процесс навигации

Сценарий взаимодействия с диалоговым ассистентом AskAI. Пользователь переходит в режим AskAI, формулирует запрос и отправляет его через поле ввода. Интерфейс отображает статус выполнения запроса, затем выводит ответ в чат. При наличии в ответе ссылок на сущности базы знаний они отображаются как интерактивные элементы и позволяют выполнить

переход к соответствующему объекту. Сценарий фиксирует ключевую для проекта задачу: не просто получить текст ответа, а встроить его в навигационный контур системы так, чтобы результат был проверяемым и полезным для последующих действий пользователя.

Компонентная диаграмма на рисунке 2.3 обосновывает структурную часть данного сценария: какие интерфейсные компоненты участвуют в отображении ответа, где выполняется преобразование данных и как обеспечивается связь с существующими механизмами навигации. Включение этой диаграммы оправдано тем, что без неё сценарий выглядел бы как «монолитное действие», тогда как фактически он реализуется кооперацией нескольких компонентов с разной ответственностью.

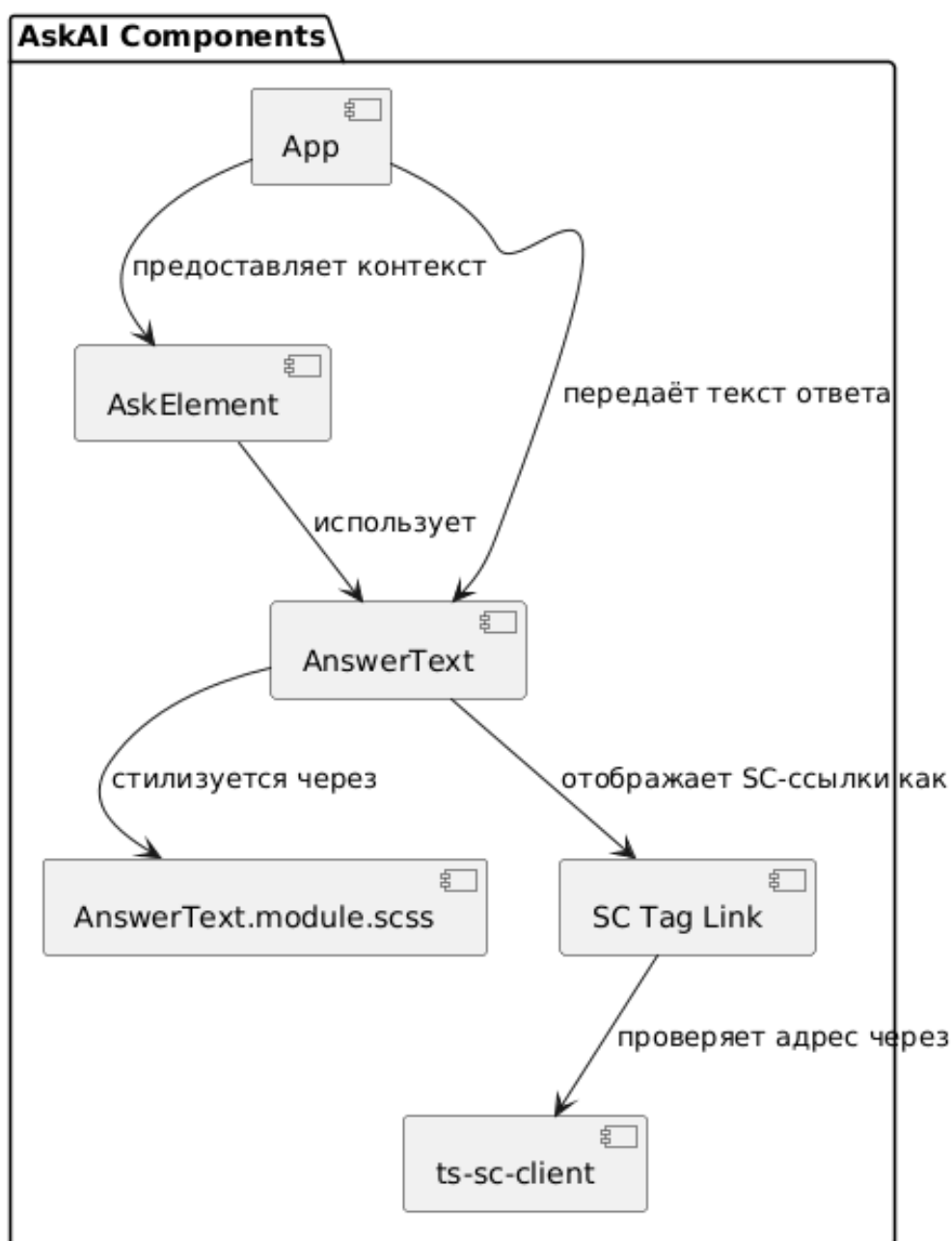


Рисунок 2.3 – Компонентная диаграмма: структура доработанного чата AskAI

Сценарий устойчивого выполнения пользовательского запроса. Пользователь выполняет типовой цикл «перейти в режим — отправить запрос — получить результат — продолжить работу». Система должна обеспечить отсутствие дублирования действий в интерфейсе, корректную индикацию загрузки, предсказуемое состояние активного режима и понятную обратную связь при ошибках. Этот сценарий включён как интеграционный, поскольку он объединяет результаты нескольких исправлений и показывает итоговое качество взаимодействия на пользовательском уровне.

Диаграмма последовательности на рисунке 2.4 подтверждает, что улучшения имеют системный характер: они затрагивают не один визуальный элемент, а согласованность переходов, событий и состояний. В данном контексте диаграмма нужна для аргументации того, что исправления не локальны, а встроены в общий жизненный цикл пользовательской операции.

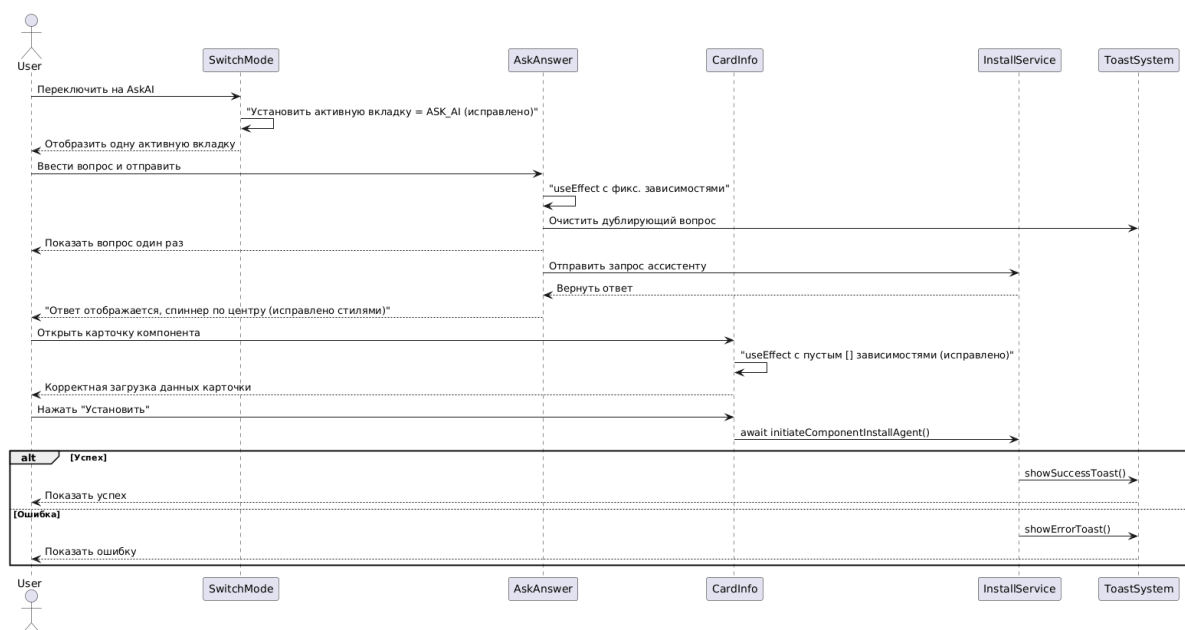


Рисунок 2.4 – Диаграмма последовательности: Исправленный интерфейс

Сценарий обработки ошибок и нестандартных ветвей. При некорректных входных данных, сетевых сбоях или конфликтных состояниях интерфейс не должен «обрываться», а должен переводить пользователя в управляемый режим: показать сообщение о проблеме, сохранить рабочий контекст и дать возможность повторить действие. Этот сценарий введён как обязательный, поскольку именно качество обработки ошибок определяет пригодность интерфейса в реальной эксплуатации.

Диаграмма деятельности на рисунке 2.5 используется для фиксации альтернативных и ошибочных ветвей, которые обычно остаются неявными в линейном текстовом описании. За счёт этого удаётся показать, что проектные решения проверяются не только на «идеальном» пути, но и на вариантах, где возникают сбои или неполные данные.

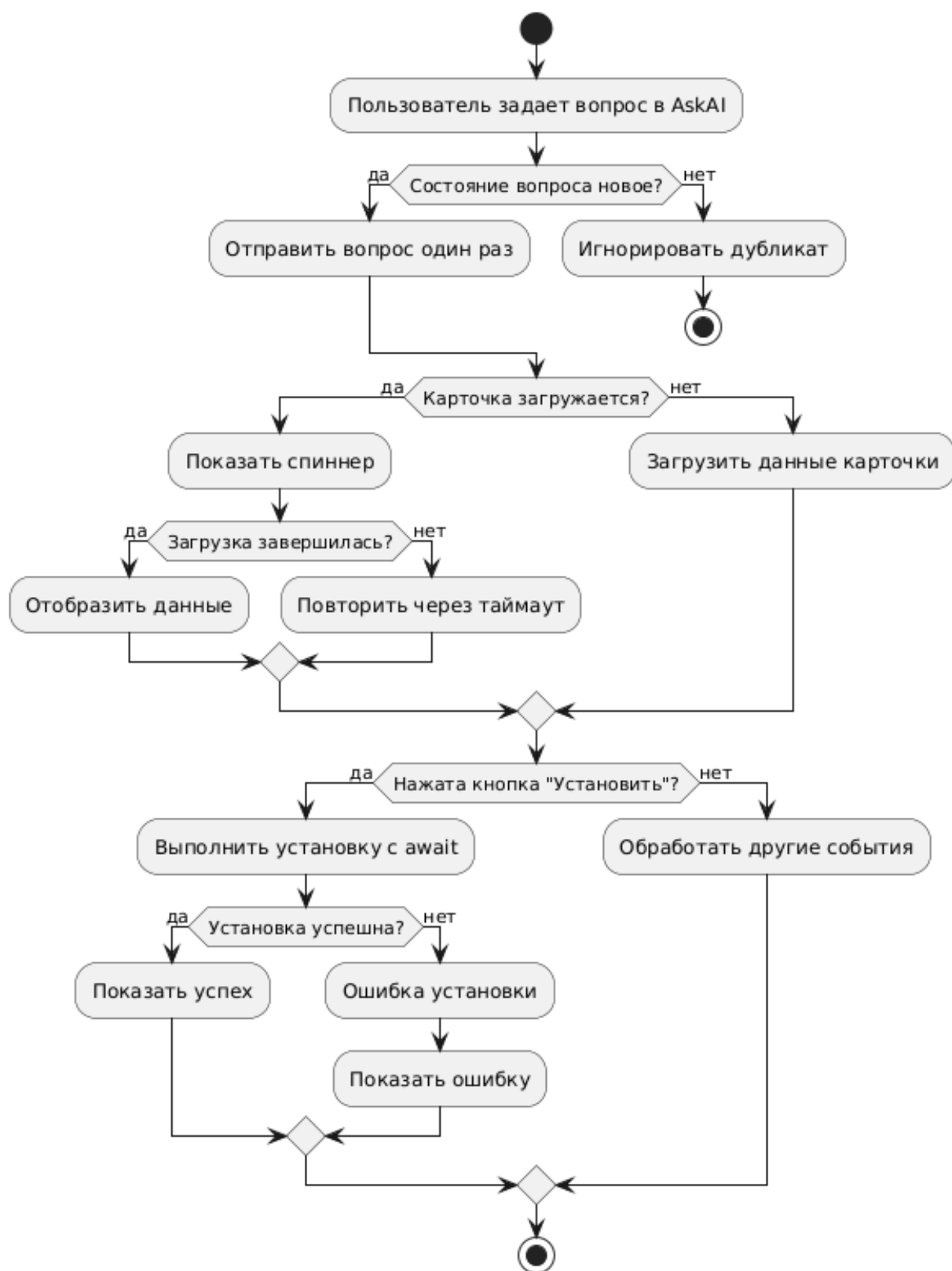


Рисунок 2.5 – Диаграмма деятельности: обработка ошибочных ситуаций в интерфейсе

Сценарий работы с библиотекой компонентов. Пользователь открывает библиотеку, получает список компонентов, выполняет поиск и фильтрацию, анализирует карточки и инициирует установку выбранного

компонента. Система должна обеспечить согласованную работу фильтра и поиска, корректное состояние элементов управления, понятную визуальную структуру карточек и прозрачную обратную связь по результату действия. Этот сценарий включён как практический индикатор того, что интерфейс поддерживает не только просмотр данных, но и управляемые пользовательские операции.

Диаграмма последовательности на рисунке 2.6 обосновывает, что изменения в библиотеке компонентов представляют собой не набор несвязанных правок, а единый управляемый сценарий с явными этапами загрузки, преобразования данных и реакции интерфейса на действия пользователя.

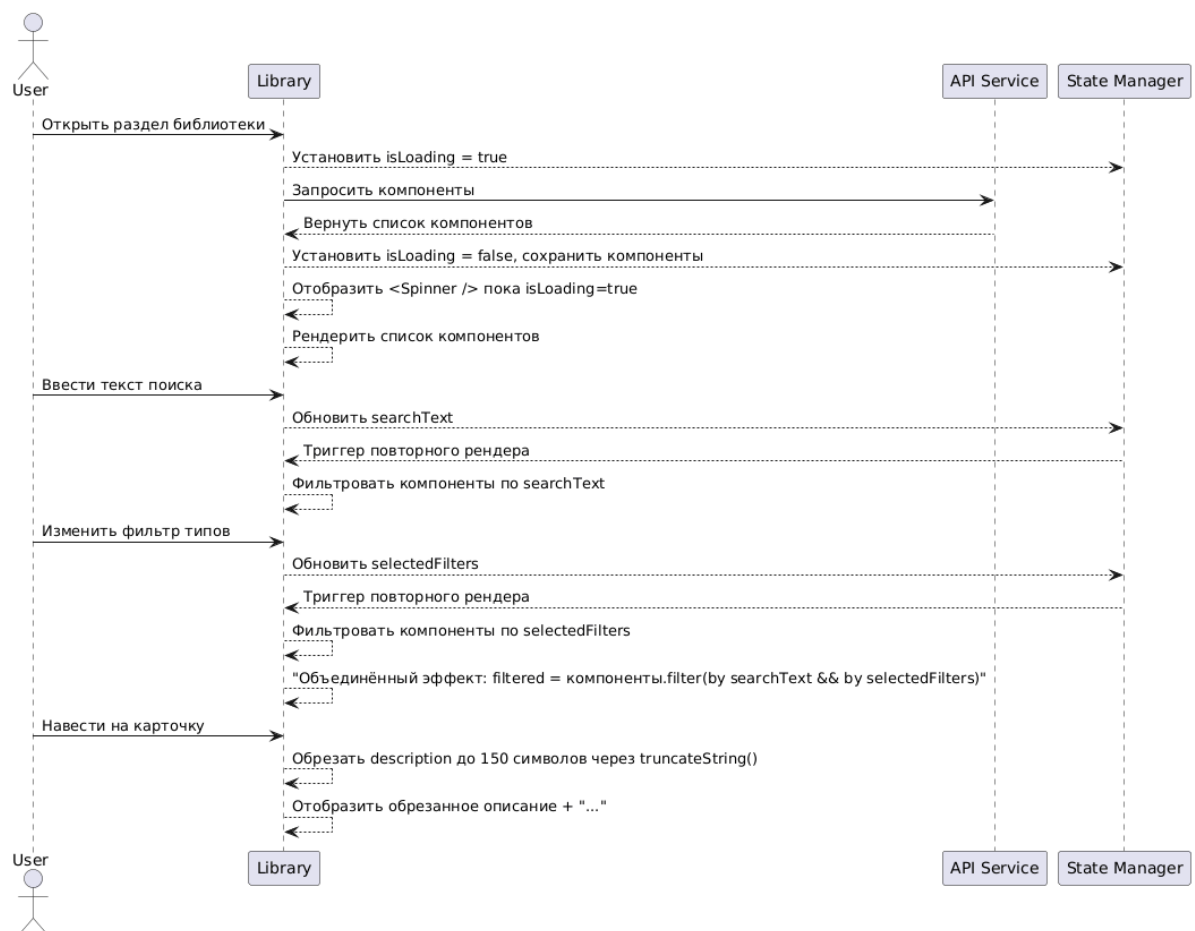


Рисунок 2.6 – Диаграмма последовательности: взаимодействие с доработанным компонентом библиотеки

2.4 Проектирование пользовательского интерфейса

Информационная структура и навигация

Проектирование пользовательского интерфейса **react-sc-web** выполнено исходя из практической цели: сократить число лишних действий при переходах между основными рабочими режимами и уменьшить вероятность потери контекста при работе с объектами базы знаний. На основе

анализа сценариев использования и фактически выполненных доработок принята трёхзонная организация экрана: глобальная навигация, переключение форматов представления и рабочая область контента. Такой подход выбран вместо более «плоской» компоновки с разнесёнными элементами управления, поскольку в плоской схеме возрастает когнитивная нагрузка: пользователь вынужден каждый раз заново определять, где находится текущий инструмент и где отображается результат действия.

В принятой структуре левая боковая панель фиксирует переходы между разделами верхнего уровня, верхняя панель обеспечивает смену режима представления данных, а центральная область концентрирует операции конкретного режима. Это решение позволяет развести постоянные и контекстные элементы управления: постоянные остаются предсказуемыми в любой точке маршрута, а контекстные отображаются только там, где они действительно нужны. Такой принцип был подтверждён в реализации: доработки *AskAI*, библиотеки компонентов, истории и переходов *SCn/SCg* проводились без пересборки общей навигационной схемы, что указывает на достаточную устойчивость выбранной структуры.

Отдельным проектным решением стало включение навигационной цепочки для разделов и связанных сущностей. Альтернативный вариант (опираться только на кнопку «назад» браузера и внутренние ссылки) был признан недостаточным, поскольку не даёт пользователю явного представления о текущем положении в иерархии знаний. Навигационная цепочка выбрана как механизм, который одновременно показывает контекст и предоставляет быстрый возврат на любой промежуточный уровень [12]. Это напрямую согласуется с задачами проекта по повышению предсказуемости интерфейса и уже подтверждено рабочими сценариями навигации.

Таким образом, информационная структура интерфейса строится не вокруг визуального удобства как самоцели, а вокруг управляемости пользовательского потока: пользователь в каждый момент понимает, где он находится, какие действия доступны в текущем режиме и куда приведёт следующий переход [13].

Макеты экранов и элементы управления

Макеты основных экранов сформированы по принципу функциональной близости [14]: элементы ввода размещаются рядом с зонами, в которых пользователь получает результат, а элементы режима — рядом с переключателями представления. В режиме *AskAI* это выражается в компактном диалоговом контуре: поле ввода, зона ответа, индикатор состояния выполнения и элементы перехода к связанным сущностям работают как единая последовательность действия. В режиме библиотеки компонентов панель поиска и фильтрации расположена до списка карточек, что поддерживает естественный сценарий «сначала сузить выбор, затем выполнить действие». В режимах просмотра семантических структур в приоритете

сохранён контекст просмотра и навигации, а не избыточное насыщение экрана управляющими кнопками.

При выборе набора элементов управления рассматривались два подхода: максимальная насыщенность экрана быстрыми действиями и более сдержанный вариант с управлением по текущему контексту. Выбран второй подход, поскольку он лучше соответствует учебно-практическому характеру системы и снижает количество ошибочных действий. В рамках реализации это решение подтвердилось тем, что часть проблем пользовательского опыта была связана именно с состояниями интерфейса, а не с нехваткой функций: исправления касались согласованности активных вкладок, корректной индикации загрузки, предсказуемости фильтрации и устойчивости сценариев при повторных действиях.

Визуальная иерархия строится на различении трёх уровней значимости: основной контент, активные действия текущего шага и служебные состояния. Основной контент (ответы, карточки, представление сущностей) занимает центральную зону внимания; активные действия выделяются формой и расположением; служебные состояния (загрузка, уведомления, ошибки) отображаются так, чтобы не скрывать результат, но оставаться заметными. Такая организация поддерживает непрерывность сценария и уменьшает число «потерянных» переходов, когда пользователь не понимает, завершилось ли действие и в каком состоянии находится система.

Принятые решения по макетам и элементам управления обеспечивают связность с предыдущими подразделами: архитектурная модель интерфейса получает на уровне *UI* конкретную форму, а ролевые сценарии — предсказуемую точку входа и завершения действия. За счёт этого проектирование пользовательского интерфейса выступает не отдельным декоративным этапом, а прямым продолжением требований и архитектурных ограничений, которые затем проверяются в разделе реализации и тестирования.

2.5 Проектирование программного интерфейса

Спецификация операций и сообщений

Проектирование программного интерфейса в **react-sc-web** выполнено как проектирование двух согласованных контуров обмена: прикладного *HTTP*-контура для пользовательских сценариев веб-приложения и специализированного контура работы с семантическими сущностями через **ts-sc-client**. Такой подход выбран по результатам анализа фактической структуры проекта и характера доработок: часть операций естественно описывается как *REST-вызовы* к серверным обработчикам, тогда как операции, завязанные на *SC*-адреса и структуру семантической сети, целесообразно оставлять в предметно-ориентированном интерфейсе, уже принятом в стеке *OSTIS*.

Для прикладного контура в проекте используются операции чтения и изменения состояния интерфейсных сценариев: получение данных для отображения, выполнение команд, работа с историей действий и обработка пользовательских запросов в чате. Для контуров, связанных с навигацией по структуре разделов и получением описаний элементов, применяются вызовы `ts-sc-client` (в частности, операции уровня `ui_menu_summary` и `ui_menu_view_get_decomposition`), поскольку в этих случаях требуется прямое соответствие семантической модели данных. Сообщения между слоями передаются в формате *JSON*, а на уровне клиентского кода приводятся к типам `TypeScript` [15], что обеспечивает предсказуемое поведение при последующей обработке в компонентах и хуках.

Альтернативой выбранной схеме мог быть полный перенос всех операций в единый *REST-контур* с сокрытием семантических вызовов за серверным фасадом. Этот вариант имеет преимущества по унификации контрактов, однако в рамках текущего проекта он был признан неоптимальным: его реализация потребовала бы расширения серверной обвязки за пределами задач интерфейсной доработки и увеличила бы объём изменений в неприоритетной для курсового проекта части. Поэтому выбран эволюционный вариант, в котором прикладные и семантические операции разделены, но согласованы по формату входных и выходных данных и по правилам обработки ошибок. С практической точки зрения такое проектное решение подтвердилось в реализации: доработки сценариев *AskAI*, навигации и библиотеки компонентов потребовали не «ломки» коммуникационного слоя, а уточнения контрактов и нормализации форматов ответов. Это позволило сохранить связность с существующей архитектурой и одновременно повысить устойчивость пользовательских сценариев к неоднозначным или частично неполным данным.

Контракты и протоколы взаимодействия

Базовым протоколом взаимодействия интерфейса с серверной частью принят *REST-over-HTTP* [16]; запросы формируются через `axios`, а ответы передаются в *JSON* с кодировкой *UTF-8*. Выбор этого протокола обусловлен его совместимостью с существующим сервером `sc-web`, прозрачностью отладки и достаточностью для целевых сценариев проекта. Применение более сложных транспортных схем (например, выделенного двустороннего протокола поверх постоянного соединения) не рассматривалось как приоритет, поскольку в реализованных сценариях доминируют запросно-ответные операции с понятными границами транзакции.

Для семантического взаимодействия применяется `ts-sc-client`, который выступает как специализированный адаптер к ядру *SC-технологии*. Это решение позволяет сохранить предметную точность операций с элементами базы знаний и не подменять семантические сущности универсальными, но более «бедными» по смыслу DTO-структурами. Контракты на клиенте

фиксируются типами данных (строки, числовые идентификаторы, массивы, флаги состояния), что снижает риск расхождения между ожидаемой и фактической структурой ответа.

Модель ошибок проектируется как двухуровневая. На первом уровне обрабатываются транспортные и протокольные ошибки (недоступность сервера, тайм-ауты, невалидные ответы), на втором — прикладные ошибки выполнения операций (включая невозможность завершения действия в текущем состоянии данных). Такое разделение выбрано осознанно: оно позволяет не смешивать проблемы канала связи и проблемы предметной логики, а значит, формировать пользователю более точную и полезную обратную связь. В интерфейсе это реализуется через централизованный механизм уведомлений (`useErrorToast`) и через проверку корректности результата до его отображения в компоненте.

В результате проектирования программного интерфейса получена схема взаимодействия, которая одновременно удовлетворяет требованиям практичности (минимум лишних изменений в серверной части), прозрачности (понятные контракты и предсказуемые форматы), и расширяемости (возможность добавлять новые сценарии без пересмотра базового обмена между слоями). Такой баланс напрямую поддерживает цель курсового проекта: повысить качество пользовательского взаимодействия с *OSTIS*, не выходя за обоснованные границы интерфейсного и интеграционного контура.

2.6 Проектирование интеллектуальных механизмов интерфейса

Интеллектуальная поддержка пользователя

В рамках проекта интеллектуальные механизмы интерфейса проектировались не как автономный модуль вывода, а как набор прикладных средств, повышающих качество взаимодействия пользователя с уже существующими семантическими сервисами *OSTIS*. Такой подход выбран сознательно: целевая задача курсового проекта заключалась в улучшении интерфейсного слоя и управляемости пользовательских сценариев, а не в разработке нового решателя. Поэтому акцент сделан на механизмах, которые интерпретируют контекст действий пользователя и подают результат в форме, пригодной для принятия следующего шага.

Ключевым решением стало использование контекстной подсказки и управляемой обратной связи в местах, где пользователь чаще всего теряет контекст: при навигации по разделам, в чате *AskAI*, при работе с библиотекой компонентов и при выполнении действий, потенциально приводящих к ошибке. Альтернативой был минималистичный интерфейс без развитых подсказок и с обработкой ошибок только на уровне статусов запроса,

однако этот вариант не соответствовал бы цели снижения порога входа и не обеспечивал бы достаточной прозрачности поведения системы для пользователя-исследователя.

На уровне практической реализации интеллектуальная поддержка выражается в том, что интерфейс не просто фиксирует действие, а интерпретирует его в текущем контексте: показывает релевантные элементы управления, сохраняет устойчивость сценария при частичных сбоях и формирует понятные уведомления вместо технических сообщений низкого уровня. В результате пользователь получает не «автоматический вывод» в строгом академическом смысле, а интеллектуализированное сопровождение взаимодействия, где система снижает неопределённость и помогает довести операцию до корректного результата.

Объяснимость и адаптация интерфейса

Проектирование объяснимости выполнено по принципу «результат должен быть проверяемым пользователем». В контуре AskAI это реализуется через связь ответа с объектами семантической сети: пользователь получает не изолированный текст, а возможность перейти к связанным сущностям и проверить контекст ответа в структуре знаний. Такой подход выбран вместо «закрытого» отображения результата, поскольку именно проверяемость и навигационная продолжимость делают ответ практическим инструментом работы, а не одноразовой подсказкой.

Адаптивность интерфейса в проекте ограничена теми механизмами, которые подтверждаются реализацией и реально влияют на рабочий поток: сохранение пользовательских настроек отображения, согласованная работа в разных режимах представления, устойчивое восстановление контекста при переходах и предсказуемая реакция на повторные действия. Рассматривался вариант более глубокой персонализации на основе накопленного профиля поведения, однако он был отложен как выходящий за границы текущего этапа: его внедрение потребовало бы отдельного слоя хранения и анализа пользовательских траекторий, что не соответствует целевому объёму интерфейсных доработок в данной работе.

Обработка ошибок проектируется как часть объяснимости: сообщение об ошибке должно не только сообщить о факте сбоя, но и сохранить пользователю понятную картину текущего состояния системы. Поэтому предпочтение отдано централизованной модели уведомлений и разделению транспортных и прикладных ошибок. Это решение повышает доверие к интерфейсу и напрямую поддерживает критерий качества результатов: пользователь видит, какие ограничения возникли, какие действия можно повторить и в каком состоянии находится текущий сценарий.

2.7 Вывод по разделу проектирования

В разделе проектирования зафиксирована и обоснована архитектура интерфейсного слоя **react-sc-web** как практический компромисс между полнотой пользовательских сценариев и ограничениями существующей платформы *OSTIS*. В качестве базового решения принята слоевая модель с разделением представления, управления состоянием, логики взаимодействия и коммуникационного контура. Выбор именно этой модели обусловлен не только общими архитектурными соображениями, но и результатами анализа проблем исходного интерфейса: обнаруженные дефекты затрагивали разные уровни системы, и их устойчивое устранение требовало изоляции ответственности между модулями, а не локальных несогласованных правок.

Проектные решения проверялись на связке «роль — сценарий — реакция системы». Для навигационных сценариев обосновано введение навигационной цепочки как механизма сохранения контекста при переходах по разделам; для диалогового сценария **AskAI** обоснована необходимость объяснимого результата через переходы к связанным сущностям; для сценариев библиотеки компонентов обоснована унификация состояния фильтрации, поиска и индикации загрузки как условия предсказуемого поведения интерфейса. Используемые BPMN- [17] и UML-диаграммы [11] в этом разделе выполняют не иллюстративную, а аналитическую функцию: они связывают процессную логику, компонентную структуру и точки возникновения ошибок с конкретными проектными решениями.

Принятый подход к программному интерфейсу — сочетание *REST-over-HTTP* [16] и специализированных вызовов **ts-sc-client** — выбран как наиболее адекватный текущему состоянию кодовой базы и задачам проекта. Альтернативный вариант полного переноса всех семантических операций в единый *REST-facade* был рассмотрен, но в рамках курсового проекта признан избыточным, поскольку увеличивал бы объём серверной переработки без сопоставимого прироста качества пользовательского взаимодействия. В результате зафиксированы контрактные принципы обмена, согласованные форматы данных и двухуровневая обработка ошибок, обеспечивающая устойчивость сценариев при частичных сбоях.

Раздел проектирования формирует целостное основание для раздела реализации: каждое существенное изменение в интерфейсе выводится из ранее сформулированной задачи, получает архитектурное объяснение и проверяется на уровне сценария использования. При этом сохраняются и границы применимости полученного решения: проект ориентирован на развитие интерфейсного слоя и интеграционного контура, не подменяя собой разработку новых алгоритмов семантического вывода или переработку ядра *OSTIS*. Такое ограничение является осознанным и методически корректным, поскольку позволяет оценивать качество результата по фактически

достигнутым целям проекта, а не по декларативно расширенному объёму работ.

3 РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

3.1 Выбор средств реализации

Языки программирования, фреймворки и библиотеки

Реализация интерфейсных доработок в проекте выполнена в технологическом стеке, уже принятом в `react-sc-web`. Базовыми средствами разработки являются `TypeScript` и `React` [18], что соответствует как архитектуре существующего клиента, так и требованиям к поддерживаемости изменений в среде, где одновременно развиваются несколько сценариев взаимодействия с базой знаний. Выбор `TypeScript` обусловлен необходимостью строго контролировать структуры данных на границах обмена между интерфейсом, серверными обработчиками и семантическим контуром; в условиях, когда часть ошибок проявлялась именно как несогласованность форматов, статическая типизация выступила не формальным преимуществом, а рабочим инструментом снижения дефектов.

Для управления общим состоянием приложения используется `Redux Toolkit`. Этот выбор сохранялся и в рамках текущей реализации, поскольку дорабатывались сценарии, затрагивающие несколько экранов и режимов одновременно: история действий, переключение форматов представления, маршрутизация, состояние запросов и отображение результата. Альтернативный вариант с локализацией состояния внутри отдельных компонентов рассматривался как более простой в краткосрочной перспективе, однако был отклонён, так как при межэкранных переходах и повторном использовании компонентов он повышает риск рассинхронизации и усложняет отладку.

Коммуникационный контур реализуется через `axios` для *HTTP*-взаимодействия и через `ts-sc-client` для операций, связанных с семантическими сущностями. Такое сочетание выбрано как практический баланс между универсальностью *REST*-запросов и предметной спецификой *SC*-операций. Полный отказ от `ts-sc-client` в пользу «чистого» *REST-контура* рассматривался, но для текущего этапа признан избыточным: он потребовал бы дополнительного слоя серверной адаптации без пропорционального выигрыша в качестве пользовательских сценариев.

Стилизация изменений выполнена в рамках принятой в проекте практики модульных стилей (*CSS Modules*) и существующей UI-базы (`ostis-ui-lib`), что позволило вносить правки, не перерабатывая базовые визуальные компоненты. Принцип минимального расширения зависимостей соблюдался целенаправленно: в проект добавлялись прежде всего новые модули и маршруты, что снижает риск регрессий и упрощает сопровождение.

Обоснование выбора средств

Критерием выбора средств в данной работе была не технологическая

новизна сама по себе, а их пригодность для решения конкретных задач, сформулированных во введении и уточнённых в разделе проектирования: устойчивое выполнение пользовательских сценариев, предсказуемость интерфейса, прозрачная обработка ошибок и сохранение согласованности между режимами работы.

С этой позиции **TypeScript** и **Redux Toolkit** обеспечили контролируемое изменение состояния и контрактов данных в сценариях, где ранее наблюдались дублирование сообщений, некорректные переходы или непоследовательное отображение результатов. **React** сохранил модульную структуру интерфейса [19] и позволил локализовать правки в конкретных компонентах без «цепной» переработки всего приложения. **axios** и **ts-sc-client** дали возможность разделить прикладной и семантический контуры обмена, что оказалось принципиально важным для корректной работы навигации, AskAI и операций с разделами/компонентами.

Альтернативные решения — миграция на другой менеджер, унификация всего обмена в одном транспортном подходе или глубокая переработка существующей *UI-библиотеки* — были признаны методически необоснованными в рамках курсового проекта. Они увеличивали бы объём инфраструктурных изменений, но не давали гарантированного выигрыша в достижении целевых пользовательских эффектов. Поэтому выбранный стек следует рассматривать как инженерно рациональный: он поддерживает расширяемость, не разрушая текущую архитектуру, и подтверждён фактической реализацией исправлений и доработок в интерфейсном слое.

Реализация пользовательского интерфейса

Реализация пользовательского интерфейса выполнена в рамках существующего приложения **react-sc-web** без изменения его базовой архитектуры, что позволило сохранить совместимость с остальными режимами *Metasystem OSTIS* [1]. Основные доработки сосредоточены не на создании отдельного «нового интерфейса AskAI», а на повышении качества уже используемых пользовательских сценариев: работе истории команд, навигации после выполнения команд из контекстного меню, управлении разделами в декомпозиции и визуальной согласованности элементов в светлой и тёмной темах. Такой подход выбран как более надёжный по сравнению с глубокой переработкой интерфейсного слоя, поскольку он снижает риск регрессий и позволяет точно улучшать проблемные места, выявленные при реальном использовании.

С точки зрения экранной логики сохранена принятая структура: пользователь работает в режимах **SCn/SCg** и боковых панелях, а изменения касаются поведения конкретных компонентов. Для панели истории скорректированы стили и интерактивные элементы: исправлено отображение кнопки удаления записи, повышена читаемость текста в тёмной теме, обновлён внешний вид кнопки очистки истории и добавлена корректная

локализация её подписи при переключении языка интерфейса. Для меню действий в декомпозиции устранена проблема прозрачного фона и приведено оформление выделения к общей визуальной системе приложения, что обеспечивает единообразное восприятие управляющих элементов.

Механизм обработки пользовательских действий реализован через существующие React-компоненты, хуки и слой запросов, с акцентом на корректность переходов и предсказуемость результата. После выполнения запроса из контекстного меню добавлена принудительная навигация в **SCn**-режим, поскольку именно он обеспечивает текстово-структурное представление результата и лучше соответствует сценарию «получить ответ и интерпретировать его». Для операций добавления и удаления разделов в декомпозиции реализована более устойчивая схема: при неуспешном *HTTP*-вызове используется резервный путь через прямое взаимодействие с *sc-metory*, благодаря чему действия пользователя сохраняются и после обновления страницы. Это решение принято как компромисс между архитектурной чистотой (только серверное *API*) и практической отказоустойчивостью интерфейса в учебном прототипе.

Отображение результатов и ошибок приведено к более корректной модели взаимодействия. Убраны избыточные уведомления в случаях, когда действие фактически сохраняется, но промежуточный ответ возвращается в неидеальном формате; сохранены только те сообщения, которые действительно помогают пользователю исправить ввод или понять причину сбоя. Для **AskAI** дополнительно снижена доля «искусственных» ответов за счёт отказа от расширенных fallback-сценариев в интерфейсе, чтобы поведение чата ближе соответствовало запросу на ответы из базы знаний. В результате пользовательский интерфейс стал более согласованным с фактической реализацией: улучшены читаемость и визуальная целостность, устранены критичные сбои в повседневных сценариях, а ключевые переходы и операции стали воспроизводимыми и понятными.

3.2 Реализация программного интерфейса

Основные операции API/протоколов. Реализация функционала **AskAI** предполагает взаимодействие клиентской части с серверной через *REST-over-HTTP* и прямые вызовы к ядру *SC-технологии*. Все операции инкапсулированы в модуле **src/api/**, где используется библиотека **axios** версии 1.4.0 с централизованным обработчиком ошибок через хук **useErrorToast**. Прямая работа с семантической сетью осуществляется синхронными вызовами методов **ts-sc-client**, возвращающими данные в *JSON-формате*, приведенном к типам **TypeScript** [15].

Операция отправки вопроса диалоговому ассистенту. Ключевой запрос для компонента **AskPage** — **POST /api/ask-ai/**. Тело запроса содер-

жит обязательное поле **question: string** с естественно-языковым запросом пользователя. Сервер обрабатывает запрос через **action_reply_to_message** ядра *SC-технологии* и возвращает ответ в виде простой строки.

Пример запроса, фиксируемый в браузере:

```
POST http://localhost:3000/api/ask-ai/
Content-Type: application/json
{
  "question": "Что такое множество?"
}
```

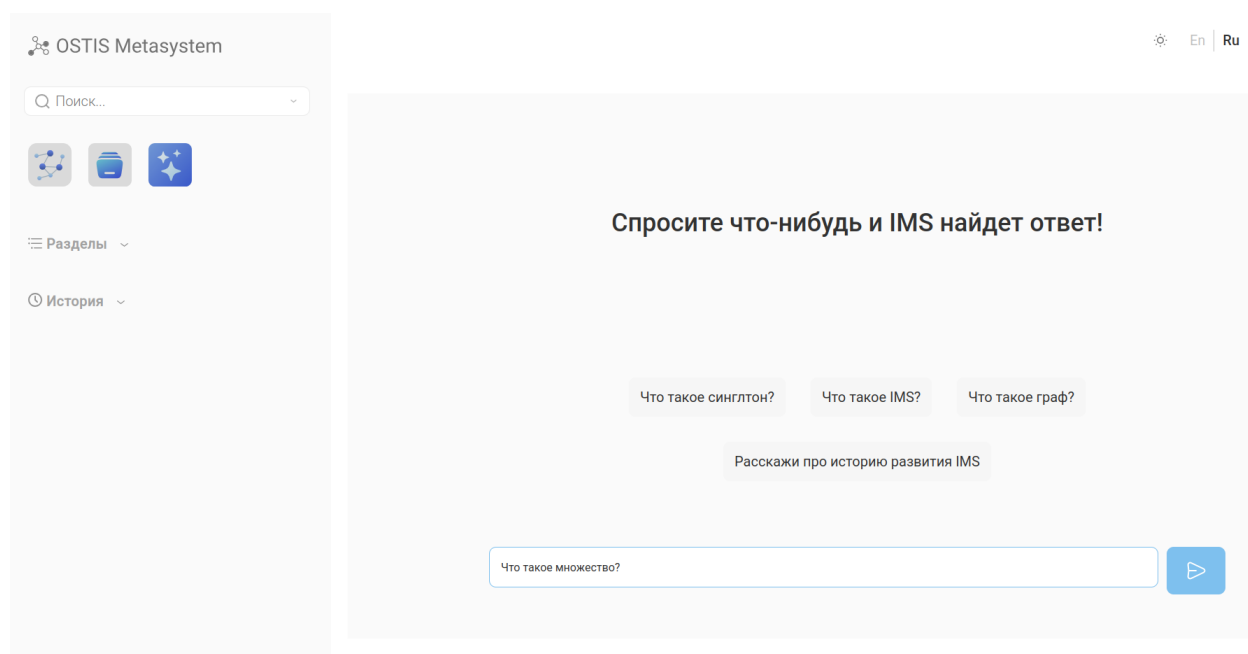


Рисунок 3.1 – Пример запроса в AskAI

Пример успешного ответа:

Под множеством понимается соединение в некое целое M определённых хорошо различимых предметов m нашего созерцания или нашего мышления (которые будут называться элементами множества M). множество – мысленная сущность, которая связывает одну или несколько сущностей в целое. Более формально множество – это абстрактный математический объект, состоящий из элементов. Связь множеств с их элементами задается бинарным ориентированным отношением принадлежность. множество может быть полностью задано следующими тремя способами: Путем перечисления (явного указания) всех его элементов (очевидно, что таким способом можно задать только конечное множество). С помощью

определяющего высказывания, содержащего описание общего характеристического свойства, которым обладают все те и только те объекты, которые являются элементами (т.е. принадлежат) задаваемого множества. С помощью теоретико-множественных операций, позволяющих однозначно задавать новые множества на основе уже заданных (это операции объединения, пересечения, разности множеств и др.). Для любого семантически ненормализованного множества существует единственное семантически нормализованное множество, в котором все элементы, не являющиеся знаками множеств, заменены на знаки множеств.

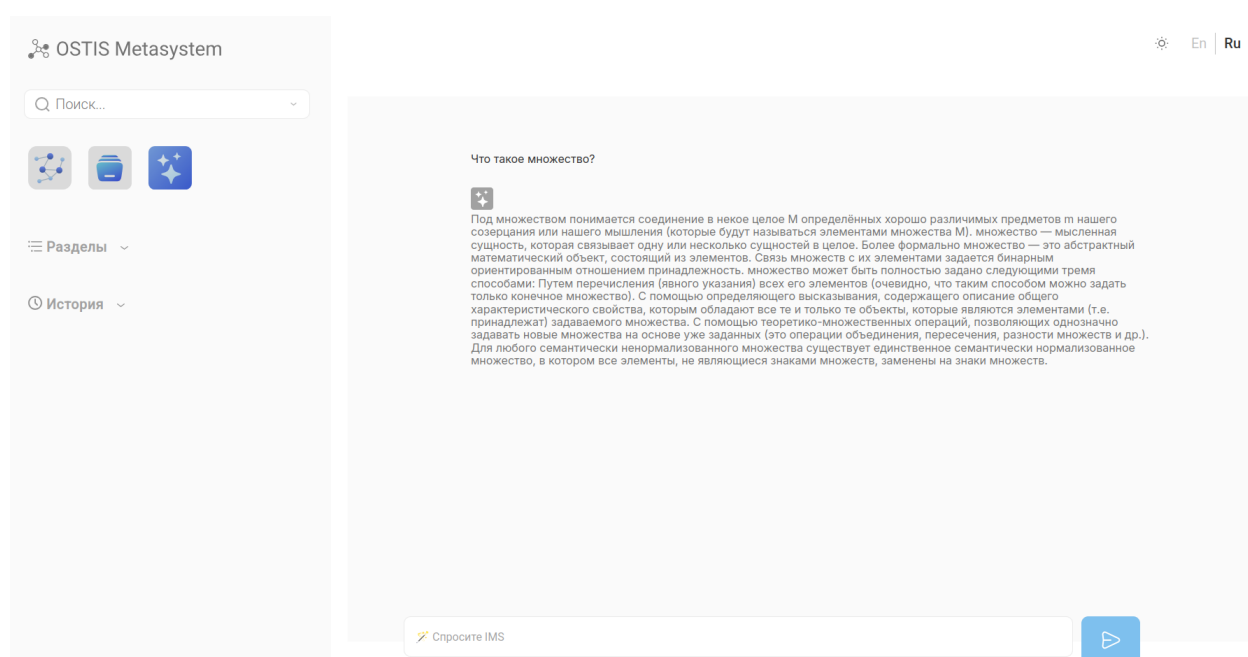


Рисунок 3.2 – Пример ответа на запрос в AskAI

При ошибке валидации (пустой вопрос) сервер возвращает код 400 и тело:

```
{
  "error": "Вы не можете отправить пустой запрос"
}
```

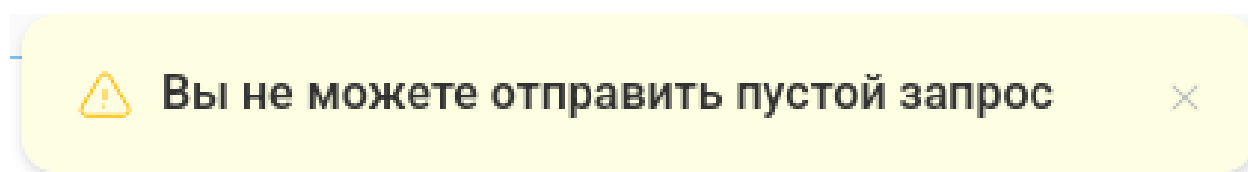


Рисунок 3.3 – Уведомление об отправлении пустого запроса AskAI

Получение описания элемента знаний. Для преобразования *SC-адресов* из ответа ассистента в кликабельные ссылки используется функция `getDescriptionByAddr` из `src/api/requests/getDescription.ts`. Она принимает числовой идентификатор элемента, выполняет операцию `ui_menu_summary` через `ts-sc-client` и возвращает строковое описание.

Функция обрабатывает случаи отсутствия элемента в *БЗ*, возвращая `null`, что приводит к отрисовке ссылки без подсказки:

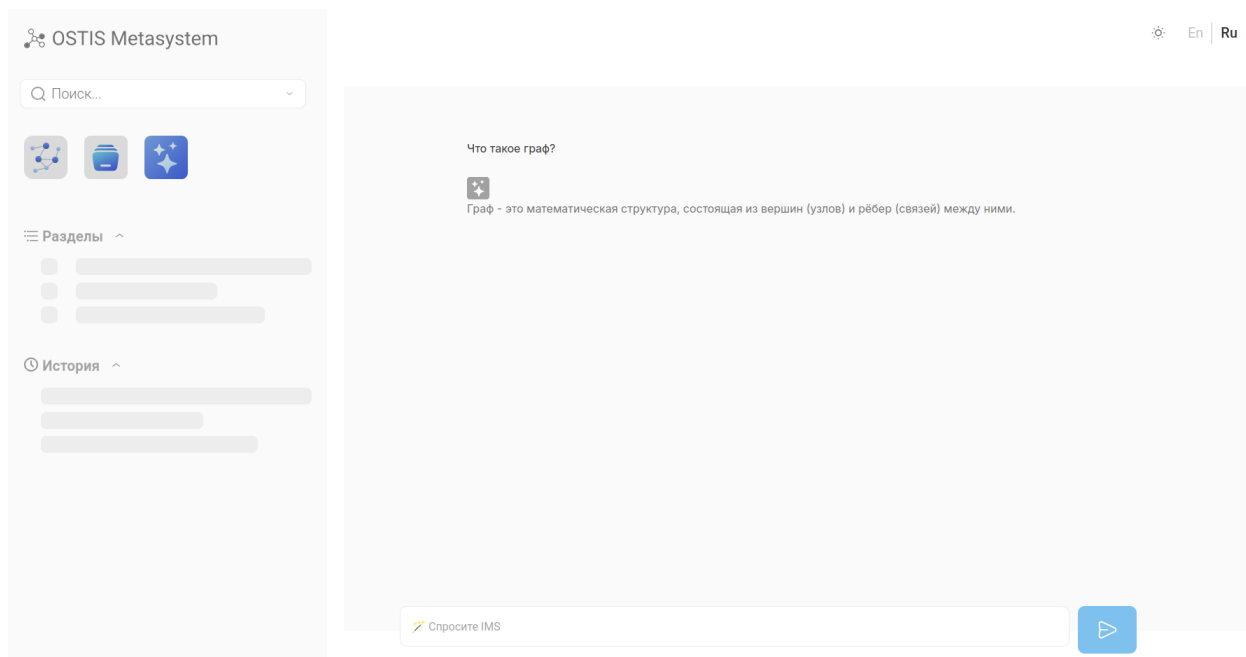


Рисунок 3.4 – Пример fallback-ответа AskAI

Механизмы обработки ошибок. Сетевые ошибки перехватываются `axios`-интерцептором и передаются в хук `useErrorToast`, отображающим уведомление в интерфейсе. Ошибки выполнения операций над семантической сетью анализируются через флаги ответа `ts-sc-client` и преобразуются в пользовательские рекомендации.

3.3 Тестирование интерфейса

Функциональное и интеграционное тестирование

Тестирование интерфейса проводилось как серия целевых проверок пользовательских сценариев, которые непосредственно связаны с реализованными доработками в `react-sc-web`. В отличие от формального автотестового контура, в данной работе применён практико-ориентированный подход: проверялись реальные цепочки действий пользователя в интерфейсе, фиксировались наблюдаемые ошибки, после чего выполнялась повторная проверка после внесения изменений. Такой формат выбран осознанно, поскольку для учебного прототипа приоритетом была проверка фактической

работоспособности ключевых сценариев взаимодействия с *SC-системой*, а не наращивание инфраструктуры тестового стенда.

Проверка сценариев истории показала и затем подтвердила устранение нескольких критичных дефектов интерфейсного уровня: некорректного отображения кнопки удаления записи, недостаточной контрастности текста в тёмной теме, а также визуально слабого оформления кнопки очистки истории. Дополнительно проверен языковой сценарий: после переключения локали подпись кнопки очистки истории корректно меняется, что подтверждает согласованность функционала с механизмом интернационализации. Отдельно верифицировано отсутствие ложного уведомления о несохранённом действии в ситуации, когда запись фактически присутствует в истории.

Для сценариев навигации выполнена проверка вызовов команд из контекстного меню в **SCg**-режиме. До доработки результат мог открываться в режиме, неудобном для интерпретации ответа; после изменения подтверждён стабильный переход к **SCn**-представлению результата. Это улучшение оценивалось не только как техническая корректность маршрутизации, но и как повышение согласованности интерфейса с пользовательской задачей «задать вопрос и прочитать структурированный ответ».

Отдельный цикл проверок выполнен для управления разделами в декомпозиции: добавление, отображение, сохранение после перезагрузки страницы и удаление. На раннем этапе воспроизводилась ошибка с созданием «пустого» элемента и последующим сообщением о неуспешном запросе. После внедрения резервного пути через **ts-sc-client** подтверждена устойчивость операций при нестабильном *HTTP*-ответе: разделы корректно создаются и удаляются, а состояние сохраняется между сессиями. Эта часть тестирования была принципиальной, так как именно она демонстрирует практическую состоятельность выбранного компромисса между «чистым» *API*-подходом и отказоустойчивостью интерфейса.

В части **AskAI** проверялись сценарии отправки корректных и некорректных запросов, получение ответа, обработка случаев отсутствия определения в базе знаний и общая предсказуемость поведения без расширенного fallback-текста. Подтверждено, что интерфейс в явном виде сообщает о ненайденном определении и не подменяет его содержательно несвязанными шаблонами, а значит результаты взаимодействия лучше соответствуют фактическому состоянию *БЗ*. Это напрямую поддерживает требование к правдоподобию и объяснимости работы интеллектуального компонента.

Оценка удобства и качества взаимодействия

Оценка качества взаимодействия выполнена методом экспертной оценки на основе результатов перечисленных сценарных проверок. По критериям удобства интерфейс после доработок стал более предсказуемым [20]: устранены визуальные дефекты управляющих элементов, улучшена читаемость в тёмной теме, снижено количество неоднозначных уведомлений, а навигация

после команд стала логически ожидаемой. По критериям «интеллектуальности» ключевым результатом является более прозрачная связь ответа с содержимым *БЗ*: пользователь видит либо подтверждённый ответ, либо корректное сообщение об отсутствии определения, что снижает риск неверной интерпретации результата.

3.4 Вывод по разделу реализации

В разделе реализации подтверждено, что поставленные задачи решены на уровне рабочего интерфейсного прототипа в составе **react-sc-web**: выполнены доработки пользовательских сценариев **AskAI**, истории, навигации после контекстных команд и управления разделами декомпозиции, а также устранены ключевые визуальные несоответствия в тёмной теме.

Принципиально важно, что изменения выполнены с сохранением существующей архитектуры *Metasystem OSTIS* [1], без избыточного расширения технологического стека, что снизило риск регрессий и обеспечило воспроизводимость результата в реальной конфигурации проекта.

Полученные результаты показывают, что интерфейс обеспечивает стабильное выполнение базовых операций: отправку запросов и получение ответов в **AskAI**, корректную обработку ошибок ввода, предсказуемое отображение результатов, а также согласованную навигацию между режимами. Существенным итогом стало повышение достоверности взаимодействия с базой знаний: в цепочке формирования ответа уменьшена роль fallback-подстановок, поэтому поведение системы в большей степени отражает фактическое содержимое *БЗ*. Для операций декомпозиции подтверждена практическая устойчивость за счёт резервного сценария через **ts-sc-client**, который компенсирует нестабильность отдельных *HTTP*-вызовов и сохраняет корректное состояние после обновления страницы.

Проведённые проверки позволили зафиксировать не только работоспособность, но и ограничения текущего решения. Прототип демонстрирует готовность к показу и использованию в учебном контуре, однако контур верификации преимущественно сценарный и ручной, что оставляет пространство для дальнейшего усиления регрессионного контроля. Логичными направлениями развития являются формализация автоматизированных проверок критичных пользовательских цепочек, расширение покрытия тестами интеграции с *sc-memory* и дальнейшее повышение объяснимости ответов **AskAI**.

В итоге реализация не сводится к частным косметическим изменениям, а формирует целостное улучшение качества взаимодействия: интерфейс стал более читаемым, предсказуемым и устойчивым, а его поведение — более согласованным с целями проекта и с реальными возможностями базы знаний *OSTIS* [1].

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта изучены подходы к проектированию интерфейсов интеллектуальных систем и уточнены требования к интерфейсному слою *Metasystem OSTIS* [1] с учётом задачи естественно-языкового взаимодействия, навигации по знаниям и устойчивой работы пользовательских сценариев. На этапе проектирования обоснована архитектура с разделением на слой представления, слой взаимодействия и коммуникационный слой; такой выбор сделан как компромисс между расширяемостью и контролируемой сложностью, поскольку альтернативы в виде монолитной логики в компонентах или, наоборот, чрезмерного выноса поведения в инфраструктурный код ухудшали либо сопровождение, либо скорость итераций при доработке интерфейса.

В части реализации подтверждено, что ключевые задачи, поставленные во введении, решены на уровне рабочего прототипа в составе **react-sc-web**. Реализованы и доведены до стабильного состояния сценарии отправки запросов в **AskAI**, отображения ответов и сообщений об ошибках, переходов после контекстных команд, работы истории действий и операций с разделами декомпозиции; дополнительно устранены критичные визуальные несоответствия в тёмной теме и улучшена локализация отдельных элементов интерфейса. Принципиальным результатом стало повышение достоверности ответов: уменьшена зависимость от fallback-формулировок, поэтому наблюдаемое поведение в большей степени отражает фактическое состояние *БЗ* а не вспомогательные шаблоны интерфейса.

Проведённое тестирование показало, что реализованные пользовательские цепочки воспроизводимы и пригодны для демонстрации в реальном режиме работы учебного прототипа. Подтверждены корректная обработка базовых и ошибочных запросов, предсказуемая навигация между режимами, устойчивое отображение результатов и практическая работоспособность механизмов добавления/удаления разделов даже при нестабильности отдельных *HTTP*-вызовов за счёт резервного взаимодействия с *sc-memory*. Тем самым достигнут основной прикладной эффект работы: снижён порог входа в использование системы, уменьшено число неоднозначных интерфейсных ситуаций и повышена объяснимость получаемых результатов за счёт более прозрачной связи ответа с содержимым знаний.

Одновременно зафиксированы ограничения текущего этапа: верификация носит преимущественно сценарный и ручной характер, а значит дальнейшее развитие целесообразно направить на формализацию регрессионного тестирования и расширение автоматизированного контроля критичных цепочек. Перспективными направлениями остаются усиление объяснимости ответов **AskAI** без возврата к содержательно слабым fallback-ответам, развитие контекстного режима диалога с учётом истории взаимодействия и

углубление интеграции с другими режимами интерфейса. В совокупности полученные результаты подтверждают достижение цели проекта: разработан и проверен согласованный с архитектурой *OSTIS* интерфейсный прототип, обеспечивающий практичное и более надёжное взаимодействие пользователя с интеллектуальной системой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Технология комплексной поддержки жизненного цикла семантически совместимых интеллектуальных компьютерных систем нового поколения / Под ред. В.В. Голенков. — Минск : Бестпринт, 2023. — 1064 с. — URL: https://libeldoc.bsuir.by/bitstream/123456789/51151/1/Tehnologii_kompleksnoi_podderjki.pdf, (дата доступа: 25.02.2026).
- [2] Голенков, В.В. Принципы построения массовой семантической технологии компонентного проектирования интеллектуальных систем / В.В. Голенков, Н.А. Гулякина // Открытые семантические технологии проектирования интеллектуальных систем = Open Semantic Technologies for Intelligent Systems (OSTIS-2011) : материалы Междунар. науч.-техн. конф., Минск, 10–12 февр. 2011 г. / Под ред. В.В. Голенков. — Минск: БГУИР, 2011. — с. 21–58.
- [3] Flowise — Build AI Agents, Visually : [сайт]. — URL: <https://flowiseai.com>, (дата доступа: 25.02.2026).
- [4] n8n.io — Flexible AI workflow automation for technical teams : [сайт]. — URL: <https://n8n.io>, (дата доступа: 25.02.2026).
- [5] Logseq — A privacy-first, open-source knowledge base : [сайт]. — URL: <https://logseq.com>, (дата доступа: 25.02.2026).
- [6] Олифер, В.Г. Сетевые технологии и протоколы: учебное пособие / В.Г. Олифер. — Санкт-Петербург : Питер, 2020. — 512 с.
- [7] Гладун, А.Я. Семантические технологии: принципы и методы / А.Я. Гладун, Ю.В. Рогушина // Управляющие системы и машины. — 2008. — № 4. — с. 33–43.
- [8] Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. — Санкт-Петербург : Питер, 2019. — 464 с.
- [9] Бэнкс, А. React и Redux: функциональная веб-разработка / А. Бэнкс, Е. Порселло. — Санкт-Петербург : Питер, 2019. — 336 с.
- [10] Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. — Санкт-Петербург : Питер, 2020. — 368 с.
- [11] Фаулер, Мартин. UML. Основы: краткое руководство по стандартному языку моделирования / Мартин Фаулер. — Санкт-Петербург : БХВ-Петербург, 2019. — 192 с.

- [12] Раскин, Дж. Интерфейс: новые направления в проектировании компьютерных систем / Дж. Раскин. — Санкт-Петербург : Символ-Плюс, 2003. — 272 с.
- [13] Норман, Д. Дизайн привычных вещей / Д. Норман. — Москва : Манн, Иванов и Фербер, 2018. — 384 с.
- [14] Купер, А. Об интерфейсе. Основы проектирования взаимодействия / А. Купер, Р. Рейман, Д. Кронин. — Санкт-Петербург : Символ-Плюс, 2018. — 688 с.
- [15] Болл, Э. Изучаем TypeScript / Э. Болл. — Санкт-Петербург : Питер, 2024. — 432 с.
- [16] Ричардсон, Л. RESTful веб-службы / Л. Ричардсон, С. Руби. — Санкт-Петербург : Символ-Плюс, 2009. — 528 с.
- [17] Сильвер, Б. BPMN. Метод и стиль / Б. Сильвер. — Москва : ДМК Пресс, 2018. — 248 с.
- [18] Стоянов, С. Изучаем React / С. Стоянов. — Санкт-Петербург : Питер, 2018. — 312 с.
- [19] Дакетт, Дж. JavaScript и jQuery: интерактивная веб-разработка / Дж. Дакетт. — Москва : Эксмо, 2017. — 640 с.
- [20] Круг, С. Не заставляйте меня думать. Веб-юзабилити и здравый смысл / С. Круг. — 3 изд. — Москва : Эксмо, 2017. — 256 с.